A Project Synopsis Report

On

PlaceMates

Submitted in partial fulfilment of the requirement of

University of Mumbai

For the Degree of

Bachelor of Technology(TY)

in

COMPUTER ENGINEERING

Submitted by

Omkar Desai (1023125)
Kalpesh Eragi (1023127)
Rohan Karkada (1023148)
Shivangi Khandelwal (1023152)

Supervised by

Mrs. Priyamvada Singh

Department of Computer Engineering
Fr. Conceicao Rodrigues Institute of Technology
An Autonomous Institute & Permanently Affiliated to University of Mumbai
Sector 9A, Vashi, Navi Mumbai - 400703

UNIVERSITY OF MUMBAI

Academic Year 2025–2026

APPROVAL SHEET

This is to certify that the project entitled

PlaceMates

Submitted by

Omkar Desai	**1023125**
Kalpesh Eragi	**1023127**
Rohan Karkada	**1023148**
Shivangi Khandelwal	**1023152**

Supervisors : _____

Project Coordinator : _____

Examiners : 1. _____

2. _____

Head of Department : _____

Date :

Place :

Declaration

We declare that this written submission for TY MiniProject Declaration entitled **PlaceMates** represent our ideas in our own words and where others' ideas or words have been included. We have adequately cited and referenced the original sources. We also declared that we have adhere to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any ideas / data / fact / source in our submission. We understand that any violation of the above will cause for disciplinary action by institute and also evoke penal action from the sources which have thus not been properly cited or from whom paper permission have not been taken when needed.

Project Group Members:

1. Omkar Desai, 1023125

_____

2. Kalpesh Eragi, 1023127

_____

3. Rohan Karkada, 1023148

_____

4. Shivangi Khandelwal, 1023152

_____

Abstract

Modern job discovery and application systems are critical in bridging the gap between candidate capabilities and industry requirements, yet most existing pipelines still rely on static resumes and keyword-based matching. Such approaches often fail to capture semantic similarity, contextual relevance, and role-specific presentation of a candidate's skills and experience. To address this limitation, several techniques are commonly employed in this domain, including TF-IDF ranking, rule-based filtering, dense vector embeddings, semantic similarity search, and retrieval-augmented generation (RAG). Among these, keyword-based and rule-driven methods are simple and efficient, but they suffer from poor semantic understanding and limited personalization, while single-pass generative systems often produce generic and ungrounded outputs. This work adopts dense embeddings for semantic matching and a retrieval-augmented, feedback-driven generation pipeline for resume optimization, as these techniques enable context-aware alignment between user profiles and job descriptions. The proposed system, **PlaceMates**, constructs a unified candidate representation from multiple data sources, performs semantic job matching using vector similarity search, and generates job-specific resumes through iterative refinement. A multi-agent draft-and-critic mechanism is used to improve quality, relevance, and ATS compatibility. Overall, the proposed framework demonstrates improved semantic alignment between candidates and job requirements, resulting in enhanced matching accuracy and the generation of highly personalized, context-aware resume content.

Contents

Abstract iii

List of Figures vii

List of Tables viii

1 Introduction 1
- 1.1 Background . . . 2
- 1.2 Motivation . . . 2
- 1.3 Aim and Objective . . . 2
- 1.4 Report Outline . . . 3

2 Study Of the System 4
- 2.1 Literature Survey . . . 5
  - 2.1.1 Review of Research Papers . . . 5
    - 2.1.1.1 Sentence-BERT: Sentence embeddings using Siamese BERT-networks [1] . . . 5
    - 2.1.1.2 Retrieval-Augmented Generation (RAG) [2] 5
    - 2.1.1.3 Matching People and Jobs [3] . . . 6
    - 2.1.1.4 SimCSE [4] . . . 6
    - 2.1.1.5 Faiss [5] . . . 7
    - 2.1.1.6 BEIR [6] . . . 7
  - 2.1.2 Research Gap . . . 8
- 2.2 Existing System . . . 9
  - 2.2.1 Overview of Existing Systems . . . 9
    - 2.2.1.1 LinkedIn Jobs . . . 9
    - 2.2.1.2 Indeed Job Search . . . 9
    - 2.2.1.3 Canva Resume Builder . . . 9
    - 2.2.1.4 Zety Resume Builder . . . 10

2.2.1.5 Semantic Job Matching Systems (Eightfold AI, HiredScore) . . . 10
2.2.1.6 AI-Based Resume Generators (Resume.io AI, Rezi AI) . . . 10
2.2.2 Comparison of Existing Systems . . . 11
2.2.3 Limitations of Existing Systems . . . 11

3 Proposed System **12**
3.1 Problem Statement . . . 13
3.2 Scope . . . 13
3.3 Proposed System . . . 14
3.3.1 User Data Ingestion . . . 14
3.3.2 Semantic Profile Representation . . . 14
3.3.3 Job Scraping and Data Pipeline . . . 14
3.3.4 Semantic Job Matching . . . 14
3.3.5 Retrieval-Augmented Resume Generation . . . 14
3.3.6 Multi-Agent Iterative Refinement . . . 15
3.3.7 Evaluation Mechanism . . . 15
3.3.8 Self-Improving Dataset . . . 15

4 Design Of the System **16**
4.1 Requirement Engineering . . . 17
4.1.1 Requirement Elicitation . . . 17
4.1.2 Software Lifecycle Model . . . 19
4.1.3 Requirement Analysis . . . 20
4.1.3.1 UML diagrams / DFDs based on the project 21
4.1.3.2 Cost Analysis (COCOMO II - Adjusted Academic Estimation) . . . 23
4.1.3.3 Hardware and Software Requirements . . 25
4.2 System architecture . . . 26
4.2.1 UI/UX Diagram . . . 26
4.2.2 Block Diagram . . . 27
4.2.2.1 Overall System Architecture . . . 27
4.2.2.2 Retrieval-Augmented Generation Pipeline 28
4.2.2.3 Multi-Agent Iterative Refinement . . . 28
4.2.2.4 Evaluation Mechanism . . . 29
4.2.2.5 Self-Improving Dataset . . . 29

5 Result and Discussion **30**
5.1 Screenshots of the System 31
5.2 Sample Code (Implementation) 33
5.2.1 Multi-Agent RAG Orchestrator 33
5.2.2 Semantic Job Matching using FAISS 34
5.2.3 Evaluation Module 34
5.3 Evaluation and Discussion 34
5.3.1 Experimental Setup 34
5.3.2 Job Matching Performance 35
5.3.3 Resume Quality 35
5.3.4 Iteration Analysis 36
5.3.5 Ablation Study 36
5.3.6 Discussion 36

6 Conclusion & Future Scope **38**
6.1 Conclusion 39
6.2 Future Scope 39

References **40**

Acknowledgement **40**

Appendix A: Timeline Chart **42**

Appendix B: Publication Details **43**

List of Figures

4.1 DFD Level 0 of PlaceMates System 21
4.2 DFD Level 1 showing system modules 21
4.3 Use Case Diagram of the System 22
4.4 Activity Diagram for Job Matching and Resume Generation 22
4.5 UI/UX Design of PlaceMates System 27
4.6 End-to-end architecture of PlaceMates system 27
4.7 Retrieval-Augmented Generation pipeline 28
4.8 Multi-agent iterative refinement loop 28
4.9 Evaluation mechanism with scoring and decision flow . . . 29
4.10 Self-improving dataset and feedback loop 29

5.1 System Interface Screenshots (Part 1) 31
5.2 System Interface Screenshots (Part 2) 32
5.3 Job Matching Performance Comparison 35
5.4 Resume Quality Across Iterations 36

6.1 Project Timeline Chart 42

List of Tables

2.1 Research Gap Summary 8
2.2 Comparison of existing systems 11

4.1 Requirement Elicitation Summary for PlaceMates System . 18
4.2 Iterative Development Plan for PlaceMates System 19
4.3 Parameters for COCOMO II Estimation 23
4.4 Adjusted Cost Drivers 24
4.5 Hardware Requirements for PlaceMates System 25
4.6 Software Requirements for PlaceMates System 26

5.1 Resume Quality Comparison 35
5.2 Ablation Results 36

Chapter 1

Introduction

1.1 Background

Modern recruitment systems operate through a pipeline that separates candidate representation from job matching. Candidate profiles are typically stored as static documents, while job matching relies on keyword-based filtering and ranking techniques. Although approaches such as TF-IDF and rule-based filtering provide computational efficiency, they fail to capture semantic relationships between skills and job requirements. Recent advancements in representation learning, particularly dense vector embeddings, have enabled semantic similarity comparison beyond surface-level keywords. Additionally, retrieval-augmented generation (RAG) has emerged as a powerful paradigm for producing context-aware and grounded textual outputs. Despite these advancements, most systems apply these techniques independently, resulting in fragmented solutions that do not fully address the problem of aligning candidate profiles with job-specific requirements.

1.2 Motivation

The primary motivation for this work arises from the limitations of static resume usage and keyword-driven job matching systems. Candidates are often required to manually tailor their resumes for each job application, which is time-consuming and prone to inconsistency. At the same time, keyword-based matching systems fail to identify relevant candidates when there is a mismatch in terminology, even if the underlying skills are aligned. This creates a disconnect between a candidate's actual capabilities and how effectively they are presented to recruiters. There is a need for a system that not only improves job matching accuracy but also dynamically adapts candidate representation based on the context of specific job roles.

1.3 Aim and Objective

The aim of this project is to design and develop a unified framework that integrates semantic job matching with dynamic, context-aware resume generation to enhance alignment between candidate profiles and job requirements.

The objectives of the proposed system are as follows:

- To construct a comprehensive candidate profile by integrating data from multiple sources such as GitHub and LinkedIn.
- To represent candidate profiles and job descriptions using dense vector embeddings for semantic similarity computation.
- To implement a semantic job matching mechanism using vector similarity search techniques.
- To develop a retrieval-augmented generation pipeline for producing job-specific, context-aware resumes.
- To incorporate a multi-agent, feedback-driven refinement mechanism for iterative improvement of generated content.
- To design an evaluation strategy for assessing matching accuracy and resume quality.
- To build a self-improving data pipeline that enhances system performance over time.

1.4 Report Outline

This report presents the design, implementation, and evaluation of the proposed system in a structured manner. Chapter 1 introduces the background, motivation, and objectives of the work. Chapter 2 reviews existing systems and highlights the limitations of traditional keyword-based and static approaches. Chapter 3 describes the proposed framework, including data ingestion, semantic profile representation, and job matching. Chapter 4 details the system architecture and key modules such as embedding, vector search, and the retrieval-augmented multi-agent pipeline. Chapter 5 presents implementation details, results, and evaluation metrics, along with a discussion of system performance. Chapter 6 concludes the work by summarizing the contributions and outlining potential directions for future enhancements. Supporting details such as the project timeline and publication scope are provided in the appendices.

Chapter 2

Study Of the System

2.1 Literature Survey

Recent research in job matching and recruitment systems has progressed from traditional keyword-based and rule-driven methods to semantic and data-driven approaches. Early systems relied on structured rules and historical data, while later work introduced machine learning to enhance recommendation accuracy. With advances in representation learning, dense embeddings enable semantic similarity between candidate profiles and job descriptions. Additionally, retrieval- and generation-based methods improve contextual understanding and content generation, reflecting a shift toward dynamic and context-aware systems.

2.1.1 Review of Research Papers

2.1.1.1 Sentence-BERT: Sentence embeddings using Siamese BERT-networks [1]

Idea mentioned in the paper: Sentence-BERT (SBERT) introduces a modification of the BERT architecture using siamese and triplet networks to generate semantically meaningful sentence embeddings. These embeddings enable efficient similarity comparison using cosine distance, making the approach suitable for large-scale semantic matching tasks.

Features:

1. Generates fixed-size sentence embeddings for efficient comparison.
2. Enables fast semantic similarity search across large datasets.
3. Reduces computational cost compared to pairwise BERT evaluation.

Technology/Tool used:

1. BERT architecture
2. Siamese and triplet networks
3. Cosine similarity measure

2.1.1.2 Retrieval-Augmented Generation (RAG) [2]

Idea mentioned in the paper: Retrieval-Augmented Generation (RAG) integrates external knowledge retrieval with sequence-to-sequence generation models. It retrieves relevant documents from a knowledge base and conditions the generated output on this information to improve factual accuracy and contextual relevance.

Features:

1. Combines retrieval and generation in a unified framework.
2. Produces context-aware and factually grounded outputs.
3. Enhances performance in knowledge-intensive tasks.

Technology/Tool used:

1. Sequence-to-sequence models
2. Dense retrievers
3. External document index

2.1.1.3 Matching People and Jobs [3]

Idea mentioned in the paper: This paper proposes a bilateral recommendation approach for job matching, where both candidate preferences and employer requirements are considered simultaneously to improve matching quality.

Features:

1. Models both user and recruiter perspectives.
2. Improves person-job fit through reciprocal matching.
3. Highlights the importance of two-sided recommendation systems.

Technology/Tool used:

1. Recommender systems
2. Preference modeling
3. Bilateral matching framework

2.1.1.4 SimCSE [4]

Idea mentioned in the paper: SimCSE introduces a contrastive learning framework for generating high-quality sentence embeddings. It leverages dropout as a minimal data augmentation technique to learn robust semantic representations.

Features:

1. Uses contrastive learning for representation improvement.
2. Supports both supervised and unsupervised settings.
3. Achieves strong performance on semantic similarity benchmarks.

Technology/Tool used:

1. BERT-based models
2. Contrastive learning
3. Dropout-based augmentation

2.1.1.5 Faiss [5]

Idea mentioned in the paper: FAISS provides an efficient framework for similarity search and clustering of dense vectors, enabling scalable nearest-neighbor retrieval in high-dimensional spaces.

Features:

1. Supports large-scale vector search efficiently.
2. Provides both CPU and GPU implementations.
3. Includes optimized indexing techniques for fast retrieval.

Technology/Tool used:

1. C++ implementation
2. Python bindings
3. GPU-accelerated indexing

2.1.1.6 BEIR [6]

Idea mentioned in the paper: BEIR introduces a comprehensive benchmark for evaluating information retrieval models across diverse datasets and domains, focusing on zero-shot performance.

Features:

1. Covers multiple retrieval tasks and domains.
2. Evaluates both sparse and dense retrieval models.
3. Emphasizes generalization across unseen datasets.

Technology/Tool used:

1. Benchmark datasets
2. BM25 baseline model
3. Dense retrieval methods

2.1.2 Research Gap

The existing literature shows that job matching, semantic embedding, and retrieval-based generation have been studied extensively as separate problems. Traditional job recommendation systems lack semantic understanding, while embedding-based approaches focus only on similarity without addressing content generation. Retrieval-augmented generation improves contextual output but is not specifically designed for candidate profiling and resume optimization. Moreover, prior systems do not integrate multi-source data ingestion, semantic matching, and iterative refinement within a single framework. Therefore, there is a need for a unified system that combines these techniques to improve both job matching accuracy and personalized resume generation.

Table 2.1: Research Gap Summary

Technical Parameter	**Current State of Technology**	**Identified Gap / Limitation**	**Reference No.**
Semantic similarity	SBERT and SimCSE use BERT-based sentence embeddings for semantic matching and similarity learning	Embeddings alone do not provide an end-to-end job matching or resume generation pipeline	[1], [4]
Retrieval quality	RAG combines retrieval with generation to improve context-aware output	Generated text may still be generic if retrieval is weak or domain-specific context is missing	[2]
Person-job matching	Bilateral recommendation models consider both candidate and employer preferences	One-sided keyword matching ignores reciprocal fit and context alignment	[3]
Similarity search	FAISS supports efficient large-scale vector search and nearest-neighbor retrieval	Exact or brute-force similarity search is slow for large embedding collections	[5]
Evaluation	BEIR provides a benchmark for retrieval models across diverse datasets	Many retrieval systems lack broad evaluation across different domains and tasks	[6]

2.2 Existing System

Current job discovery and resume-building platforms operate using largely independent workflows for profile representation, job matching, and document generation. While these systems provide useful functionalities, they exhibit several limitations in terms of semantic understanding, personalization, and adaptability.

2.2.1 Overview of Existing Systems

2.2.1.1 LinkedIn Jobs

Idea: LinkedIn Jobs provides a large job search platform with filters for skills, location, and experience, along with job alerts and application features.

Limitations:

- Matching is mostly keyword and filter based.
- It does not provide semantic alignment with candidate profiles.
- It does not generate job-specific resume content.

2.2.1.2 Indeed Job Search

Idea: Indeed allows users to search and apply for jobs using keywords, title, company, and location-based filters.

Limitations:

- It depends heavily on exact keywords.
- It lacks profile adaptation for specific roles.
- It does not support integrated resume optimization.

2.2.1.3 Canva Resume Builder

Idea: Canva offers a template-based resume builder with drag-and-drop editing and downloadable resume formats.

Limitations:

- It is mainly design focused.
- It does not analyze user skills or projects deeply.
- It does not tailor content to a target job.

2.2.1.4 Zety Resume Builder

Idea: Zety provides guided resume creation using templates, sample content, and ATS-friendly formatting.

Limitations:

- It relies on predefined templates and prompts.
- It does not use verified profile data.
- It lacks iterative content refinement.

2.2.1.5 Semantic Job Matching Systems (Eightfold AI, HiredScore)

Idea: These systems use AI-driven embeddings and contextual analysis to match candidate profiles with job roles based on skills, experience, and potential rather than exact keywords.

Limitations:

- Primarily focused on matching and ranking, not resume/content generation.
- Designed for enterprise use and depend heavily on structured HR/talent data.
- Limited integration of multi-source candidate data such as GitHub or external portfolios.

2.2.1.6 AI-Based Resume Generators (Resume.io AI, Rezi AI)

Idea: These tools use language models to generate ATS-friendly resume content based on user input and job descriptions.

Limitations:

- Outputs may be generic without strong grounding in verified user data.
- Focus mainly on resume creation, not job matching or semantic alignment.
- Lack iterative, feedback-driven refinement mechanisms for improving content quality.

2.2.2 Comparison of Existing Systems

System	Type	Resume Builder	Job Matching	AI/NLP-Based	Bilateral Matching	Key Limitation
LinkedIn Jobs	Job Portal	No	Yes	No	No	Keyword-based matching and low semantic fit
Indeed Job Search	Job Portal	No	Yes	No	No	Depends on filters and exact keywords
Canva Resume Builder	Resume Builder	Yes	No	No	No	Template-based with no job adaptation
Zety Resume Builder	Resume Builder	Yes	No	Yes	No	Resume-focused, limited role-specific personalization
Semantic Job Matching Systems	Semantic Matching	No	Yes	Yes	No	Matching only, no resume generation
AI-Based Resume Generators	AI Resume Generator	Yes	No	Yes	No	Generic output without strong grounding

Table 2.2: Comparison of existing systems

2.2.3 Limitations of Existing Systems

The major limitations of existing systems are summarized as follows:

- Lack of semantic understanding in job matching, resulting in irrelevant recommendations.
- Static representation of candidate profiles without adaptation to specific job roles.
- Absence of integration between profile analysis, job matching, and resume generation.
- Limited personalization and inability to leverage multi-source data such as GitHub and LinkedIn.
- Lack of iterative refinement mechanisms for improving generated content.

These limitations highlight the need for a unified system that integrates semantic matching, dynamic profile representation, and feedback-driven content generation, as proposed in this work.

Chapter 3

Proposed System

3.1 Problem Statement

Existing job discovery systems rely on keyword-based matching and static resumes, which fail to capture semantic relationships between candidate profiles and job descriptions. This results in poor alignment, low personalization, and generic resume submissions. Furthermore, current approaches lack integration between profile analysis, job matching, and content generation, and do not support iterative refinement. Therefore, there is a need for a unified system that performs semantic job matching and generates job-specific resumes using retrieval-augmented and feedback-driven techniques.

3.2 Scope

The proposed system aims to provide an integrated solution for improving the job application process through semantic job matching and dynamic resume generation. The system focuses on collecting and processing user data from multiple sources such as GitHub and LinkedIn to construct a comprehensive and structured candidate profile. It leverages dense vector embeddings to represent both candidate profiles and job descriptions, enabling context-aware similarity matching beyond traditional keyword-based approaches.

The scope of the system includes the implementation of a retrieval-augmented generation pipeline to produce job-specific, ATS-friendly resumes. The system also incorporates a multi-agent, feedback-driven refinement mechanism to iteratively enhance the quality, relevance, and alignment of generated content with job requirements. Additionally, the system supports efficient job ranking using vector similarity search and filtering techniques.

The system also incorporates an evaluation mechanism to assess resume quality and a self-improving dataset that stores high-quality generated outputs for future reuse. This enables continuous improvement in retrieval and generation performance over time.

3.3 Proposed System

The proposed system, *PlaceMates*, is designed as a unified framework that integrates semantic job matching with dynamic, job-specific resume generation. The system combines multi-source data ingestion, semantic retrieval, retrieval-augmented generation, and iterative refinement within a single pipeline.

The overall workflow consists of the following modules:

3.3.1 User Data Ingestion

The system collects user data from multiple sources such as GitHub and LinkedIn. This data includes projects, skills, experience, and education. The collected information is processed and structured to form a comprehensive candidate profile.

3.3.2 Semantic Profile Representation

The structured profile is converted into a semantic representation using dense vector embeddings. This enables contextual understanding of user skills and experience beyond keyword matching.

3.3.3 Job Scraping and Data Pipeline

Job data is collected using an automated scraping pipeline and stored in a database. The job descriptions are preprocessed and converted into embedding vectors for efficient retrieval.

3.3.4 Semantic Job Matching

The system performs similarity search between candidate and job embeddings using vector similarity techniques. A two-stage retrieval process is used, where semantic matching is followed by filtering based on relevance criteria such as recency and preferences.

3.3.5 Retrieval-Augmented Resume Generation

For each matched job, the system generates a job-specific resume using a retrieval-augmented generation approach. Relevant resume examples are retrieved and used as context to guide generation.

3.3.6 Multi-Agent Iterative Refinement

A multi-agent framework consisting of a retriever, drafter, and evaluator is used to iteratively improve resume quality. The evaluator provides structured feedback, which is used to refine the resume until a predefined quality threshold is achieved.

3.3.7 Evaluation Mechanism

The system evaluates both job matching and resume quality using metrics such as precision, recall, and ATS-based scoring. This ensures that generated outputs meet quality standards.

3.3.8 Self-Improving Dataset

High-quality generated resumes are stored and reused in future retrieval processes. This creates a feedback loop that continuously improves system performance over time.

Overall, the proposed system provides an end-to-end intelligent pipeline that enhances job matching accuracy and generates personalized, context-aware resumes tailored to specific job roles.

Chapter 4

Design Of the System

4.1 Requirement Engineering

Requirement Engineering is a systematic process of identifying, analyzing, documenting, and managing the requirements of a software system. It ensures that the developed system satisfies user needs and operates efficiently. For the proposed system, PlaceMates, requirement engineering is carried out through structured phases including elicitation, analysis, and validation.

4.1.1 Requirement Elicitation

Requirement elicitation involves identifying and gathering the functional and non-functional needs of users and stakeholders. For the proposed system, inputs were collected through analysis of existing job platforms, observation of job search workflows, and identification of challenges faced by candidates during resume preparation and job matching.

The key requirements identified during the elicitation process are summarized in the table below.

Table 4.1: Requirement Elicitation Summary for PlaceMates System

Requirement Description	**Requested By**	**Category**	**Priority**	**Acceptance Criteria**
Semantic job matching based on contextual similarity instead of keyword filtering.	Candidate	Functional	High	System should recommend jobs with high contextual relevance based on profile and job description.
Automated generation of job-specific resumes.	Candidate	Functional	High	Resume should be generated dynamically based on selected job requirements.
Integration of multi-source profile data (GitHub, LinkedIn, manual input).	Candidate	Functional	High	System should successfully extract and combine data from multiple sources into a structured profile.
Context-aware resume optimization using job descriptions.	Candidate	Functional	High	Generated resumes should reflect relevant skills and align with job requirements.
Reduction of manual effort in resume customization.	Candidate	Operational	High	User should be able to generate a tailored resume with minimal manual input.
Iterative improvement of resume quality through evaluation feedback.	System	Functional	Medium	Resume quality score should improve across iterations until threshold is achieved.
Efficient retrieval of relevant job data using semantic search.	System	Technical	High	Retrieval system should return relevant job results within acceptable response time.
Secure handling and storage of user data (no raw sensitive data exposure).	System	Technical	High	Sensitive data should not be stored in raw form; only processed or embedded representations should be used.

4.1.2 Software Lifecycle Model

The proposed system follows the **Iterative Software Development Model**, where the system is built and refined in multiple stages. Each iteration focuses on a specific module and incorporates improvements based on feedback from previous stages. This approach enables gradual development, early validation, and continuous enhancement of system quality.

Table 4.2: Iterative Development Plan for PlaceMates System

Iteration / Phase	**Activities Carried Out**	**Outcome / Deliverable**
Iteration 1: Requirement Analysis	Analyzed job workflows and identified system limitations. Defined requirements and key features.	Defined problem scope and requirement specification.
Iteration 2: System Design	Designed architecture, DFDs, and workflows. Planned module integration.	Completed system design and architecture blueprint.
Iteration 3: Profile Processing	Developed multi-source data ingestion and structured candidate profiles.	Functional profile construction module.
Iteration 4: Semantic Job Matching	Implemented embedding-based matching and job retrieval. Tested performance.	Working semantic matching module with improved relevance.
Iteration 5: Resume Generation (RAG)	Developed retrieval-augmented resume generation with contextual inputs.	System generating job-specific resumes.
Iteration 6: Refinement & Evaluation	Implemented refinement loop and evaluation mechanism. Improved output quality.	Enhanced resumes with measurable quality improvement.
Iteration 7: Testing & Validation	Performed unit and integration testing. Fixed issues and optimized system.	Validated system with stable and reliable outputs.

Advantages: The iterative model allowed gradual development and continuous improvement of the system. Each phase produced a working component that could be tested and refined. This approach reduced development risk, improved system quality, and ensured that issues were identified early and resolved effectively.

4.1.3 Requirement Analysis

Requirement analysis refines the collected requirements into structured specifications that define system behavior, module interactions, and constraints.

User Roles

- **Candidate**: Provides profile data, views job recommendations, and generates resumes.
- **System**: Processes data, performs matching, and generates optimized outputs.

Functional Requirements

- Process multi-source profile data.
- Perform semantic job matching.
- Generate job-specific resumes (RAG).
- Refine resumes iteratively using feedback.
- Evaluate resume quality.
- Store and reuse high-quality outputs.

Non-Functional Requirements

- **Performance**: Fast job retrieval and resume generation.
- **Scalability**: Handles increasing users and data.
- **Security**: No raw sensitive data storage.
- **Reliability**: Consistent output across iterations.
- **Usability**: Minimal manual effort required.

Module Interaction

- Profile data → semantic matching → job context.
- Job context → resume generation → refinement loop.
- Evaluation determines acceptance or further refinement.
- High-quality outputs stored for reuse.

Outcome

Defined clear system functionality, constraints, and data flow, forming the basis for system design and implementation.

4.1.3.1 UML diagrams / DFDs based on the project

The system can be represented using Data Flow Diagrams (DFDs) and UML diagrams to illustrate the flow of data and interaction between components.

DFD Level 0:

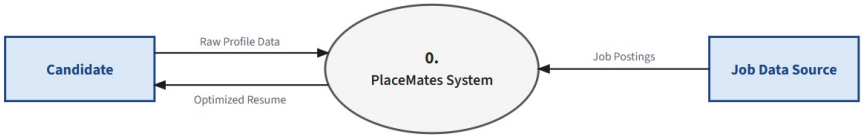


Figure 4.1: DFD Level 0 of PlaceMates System

DFD Level 1:

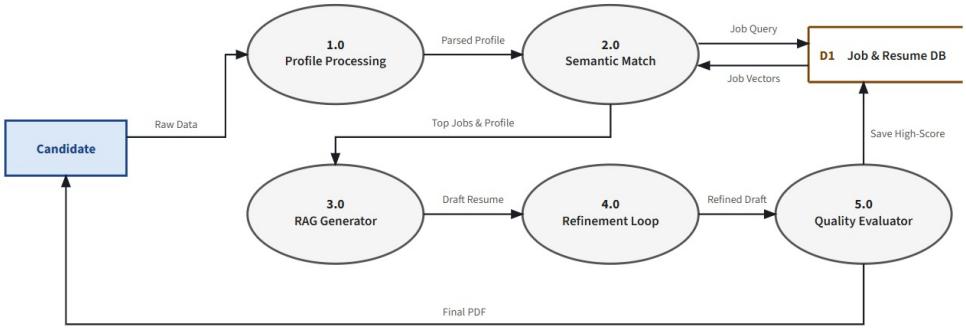


Figure 4.2: DFD Level 1 showing system modules

Use Case Diagram:

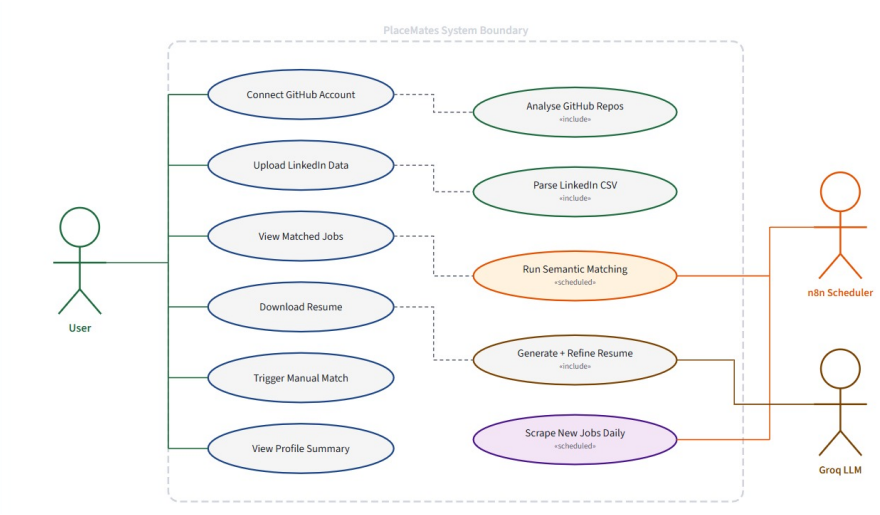


Figure 4.3: Use Case Diagram of the System

Activity Diagram:

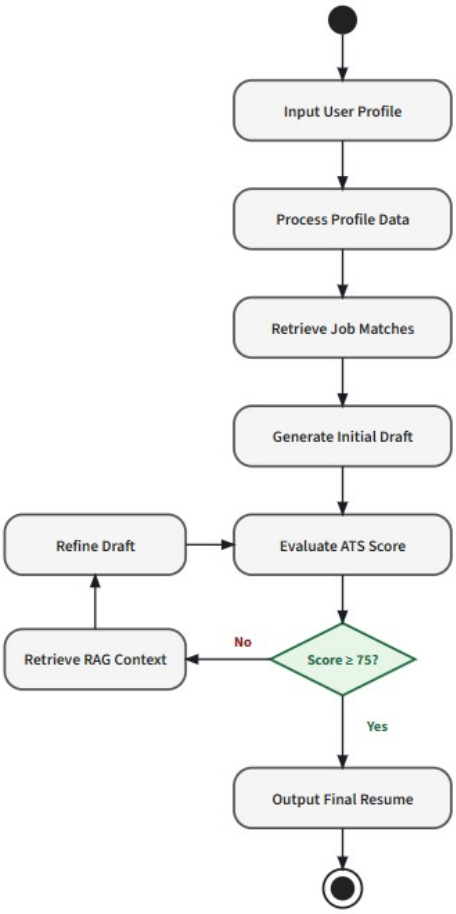


Figure 4.4: Activity Diagram for Job Matching and Resume Generation

Description:

- DFD Level 0 provides an overview of system interaction with users and external data sources.
- DFD Level 1 illustrates internal processes such as data ingestion, semantic matching, and resume generation.
- The Use Case Diagram represents interactions between users and the system functionalities.
- The Activity Diagram shows the workflow of job matching and resume generation.

4.1.3.2 Cost Analysis (COCOMO II - Adjusted Academic Estimation)

The cost estimation for the proposed system is performed using the **COCOMO II Model**. Since the project is developed in an academic environment, adjusted interpretation is applied to reflect realistic effort within a student development scope.
The general COCOMO II formula is:

$$Effort(Person - Months) = A \times (Size)^B \times EAF$$

The project size is estimated at approximately **30 KLOC**, including frontend, backend, and supporting AI components. Due to the integration of multiple technologies and modular design, the system is considered moderately complex.

Parameter	**Value**	**Remarks**
Estimated Size (KLOC)	30.0	Based on total project codebase.
Scale Factor (B)	1.08	Moderate complexity with structured modules.
EAF	1.05	Academic environment with fewer constraints.
Constant (A)	2.94	Standard COCOMO II constant.

Table 4.3: Parameters for COCOMO II Estimation

Using the standard COCOMO II calculation:

$$Effort = 2.94 \times (30)^{1.08} \times 1.05$$

$$Effort \approx 121 Person - Months$$

The above value represents an industrial-scale estimation. However, since this system is implemented as an academic project with controlled scope, partial feature implementation, and the use of modern development tools, the practical effort is significantly lower.

Adjusted Academic Estimation:

- Total estimated effort: **20–24 person-months**
- Effort per student (4 members): **5–6 months**
- Estimated duration: **5–7 months**

Summary of Key Cost Drivers:

Cost Driver	**Rating**	**Remarks**
Product Complexity	Moderate	Multi-module system with AI components.
Platform Complexity	Moderate	Integration of web and ML components.
Team Experience	Moderate	Familiar with core technologies.
Tool Support	High	Use of modern frameworks and APIs.
Schedule Constraint	Moderate	Academic timeline constraints.

Table 4.4: Adjusted Cost Drivers

Interpretation:

- Industrial estimation: **121 person-months**
- Adjusted academic effort: **20–24 person-months**
- Project duration: **5–7 months**
- Effort per student: **5–6 months**

Since this is an academic project, no direct monetary labor cost is involved. The system primarily uses open-source tools and limited cloud resources, making it cost-efficient while demonstrating significant development effort.

4.1.3.3 Hardware and Software Requirements

The proposed system requires moderate hardware and modern software tools to support multi-agent processing, semantic retrieval, and web-based interaction. The following tables summarize the required specifications.

Hardware Requirements

Component	**Specification**	**Remarks**
Processor	Intel i5 / AMD Ryzen 5 or higher	Supports concurrent execution of multiple services (frontend, backend, AI modules).
RAM	Minimum 8 GB (16 GB recommended)	Required for running Docker containers, Node.js services, and ML models simultaneously.
Storage	SSD with 5 GB+ free space	Ensures fast dependency loading, vector storage, and system performance.
Internet Connectivity	Stable broadband connection	Required for API calls (LLMs, job data) and cloud database access.

Table 4.5: Hardware Requirements for PlaceMates System

Software Requirements

Component	Specification	Remarks
Runtime Environment	Node.js 18+, npm 9+	Used for backend services and frontend (Next.js).
Programming Language	Python 3.10+	Handles embedding generation and AI processing.
Frameworks	Next.js, Express.js	Supports frontend UI and backend API development.
Databases	PostgreSQL, FAISS / ChromaDB	Used for structured storage and vector-based semantic retrieval.
Workflow Engine	n8n	Manages automation and multi-agent pipeline execution.
Containerization	Docker Desktop	Runs isolated services (AI models, workflows).
Version Control	Git	Maintains source code and collaboration.
External APIs	Groq / Gemini, GitHub, LinkedIn, Apify	Provides LLM inference, profile data, and job extraction.
Cloud Services	Cloudinary, NeonDB	Handles file storage and serverless database hosting.
Development Tools	VS Code	Used for coding, debugging, and project management.
Browser	Chrome / Edge / Safari (latest)	Required for frontend interaction and workflow UI.

Table 4.6: Software Requirements for PlaceMates System

4.2 System architecture

4.2.1 UI/UX Diagram

The user interface of the system is designed to provide a simple and intuitive workflow for users to input profile data, view job recommendations, and generate resumes.

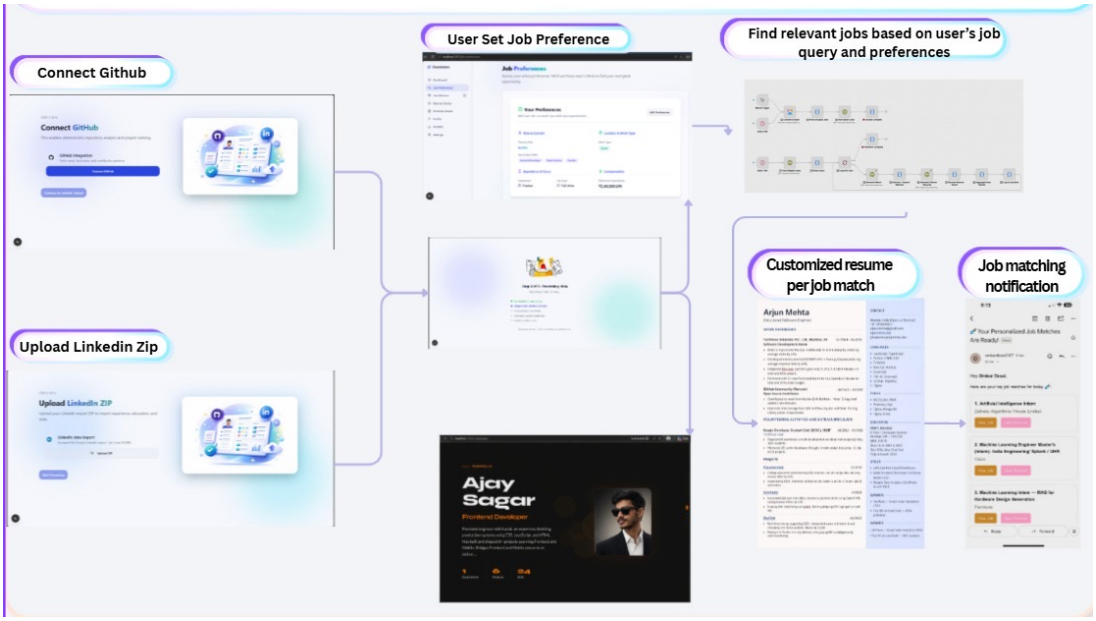


Figure 4.5: UI/UX Design of PlaceMates System

4.2.2 Block Diagram

The proposed system consists of multiple interconnected modules forming an end-to-end pipeline. The overall workflow, along with detailed sub-modules, is illustrated through the following diagrams.

4.2.2.1 Overall System Architecture

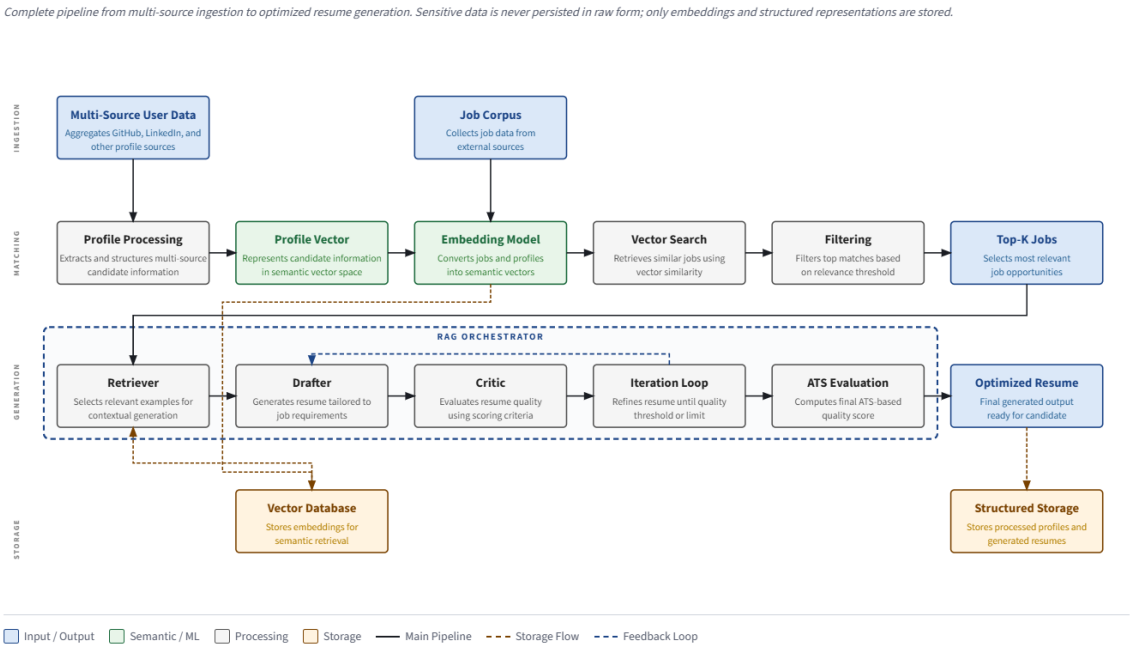


Figure 4.6: End-to-end architecture of PlaceMates system

The above diagram represents the complete pipeline, including multi-source data ingestion, semantic job matching, retrieval-augmented resume generation, and feedback-driven

refinement.

4.2.2.2 Retrieval-Augmented Generation Pipeline

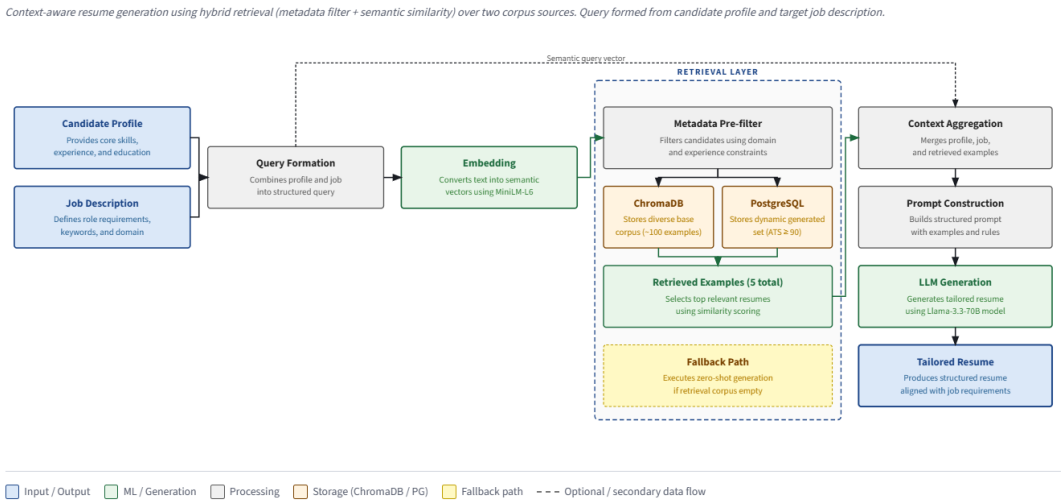


Figure 4.7: Retrieval-Augmented Generation pipeline

This module constructs queries from candidate profiles and job descriptions, retrieves relevant examples using semantic similarity, and generates job-specific resumes using contextual grounding.

4.2.2.3 Multi-Agent Iterative Refinement

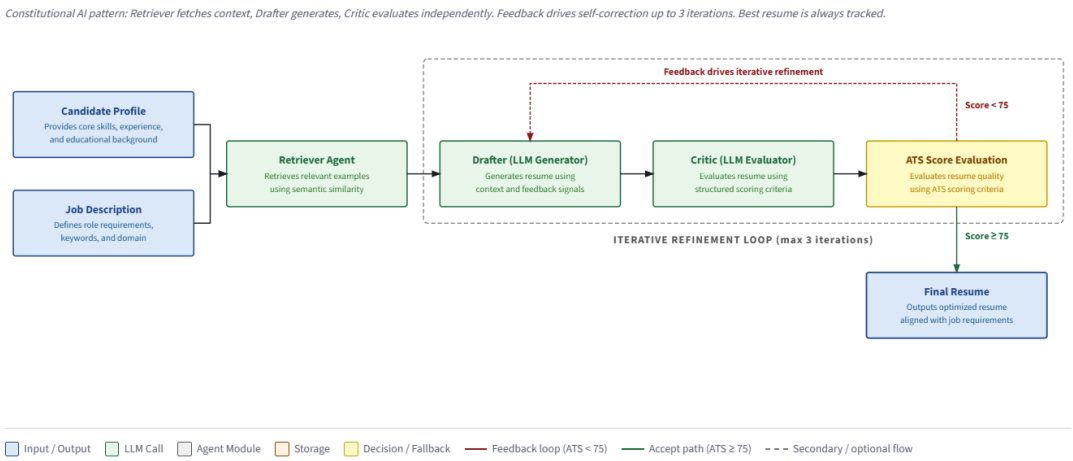


Figure 4.8: Multi-agent iterative refinement loop

This process involves three agents: retriever, drafter, and evaluator. The system iteratively improves the resume using feedback until a predefined quality threshold is achieved.

4.2.2.4 Evaluation Mechanism

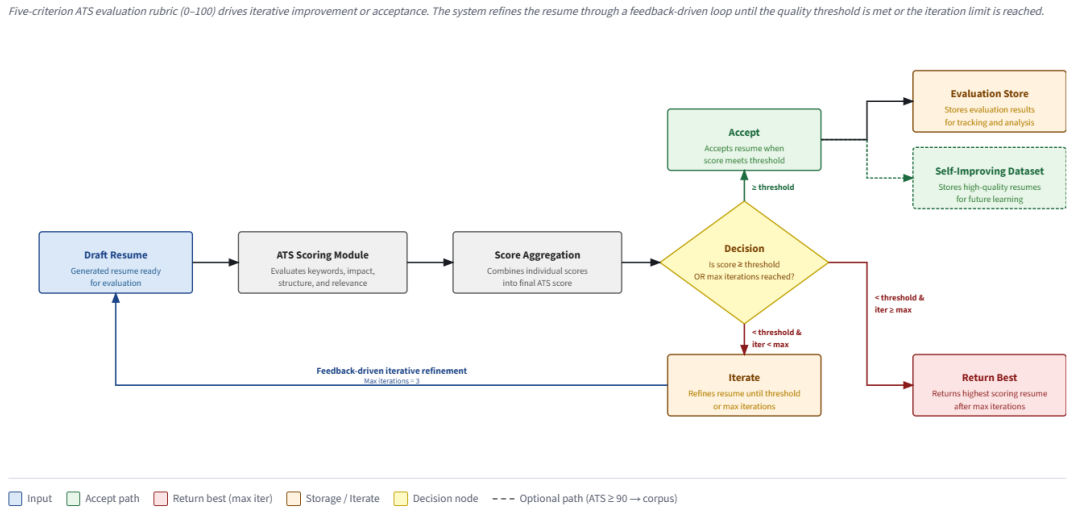


Figure 4.9: Evaluation mechanism with scoring and decision flow

The evaluation module computes an ATS-based score and determines whether to accept the resume or continue refinement based on threshold conditions.

4.2.2.5 Self-Improving Dataset

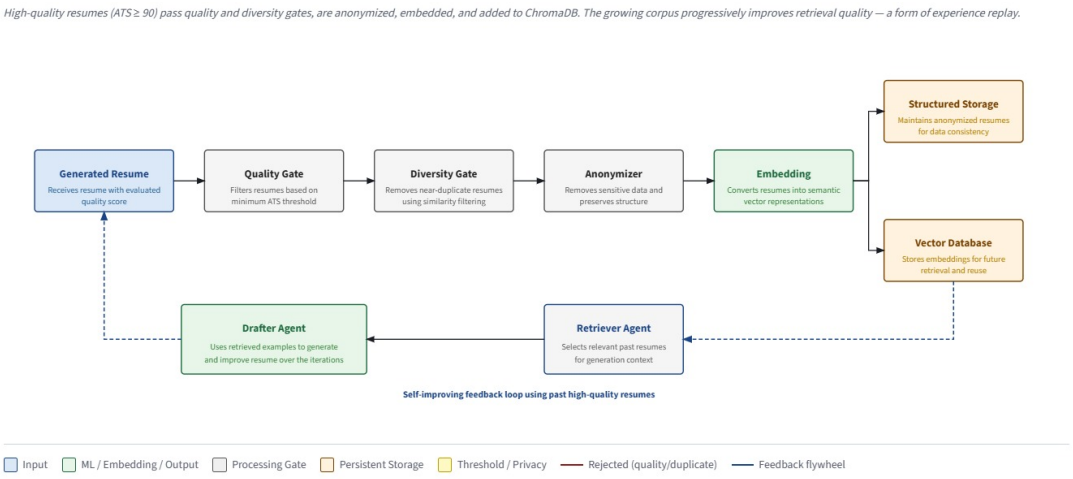


Figure 4.10: Self-improving dataset and feedback loop

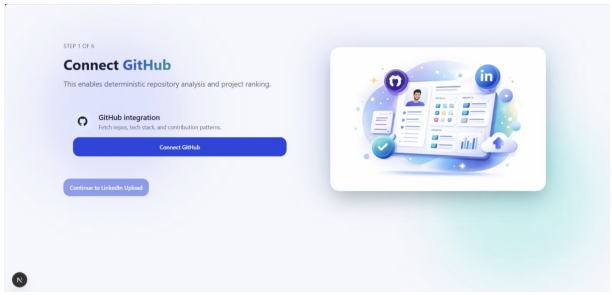
High-quality resumes are filtered, anonymized, embedded, and stored in a vector database. These are reused in future iterations, enabling continuous system improvement.

Chapter 5

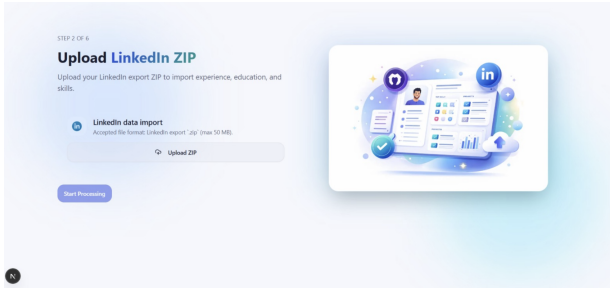
Result and Discussion

5.1 Screenshots of the System

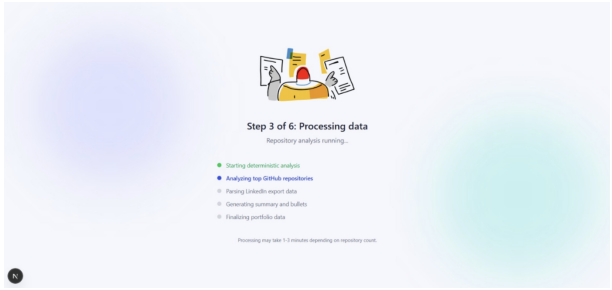
The following screenshots illustrate the implementation and user interface of different modules of the PlaceMates system.



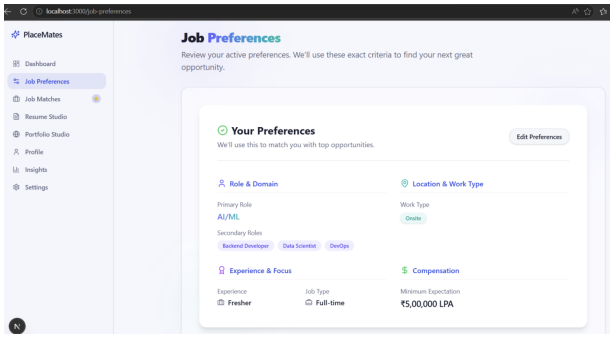
(a) GitHub Integration Interface



(b) LinkedIn Data Upload

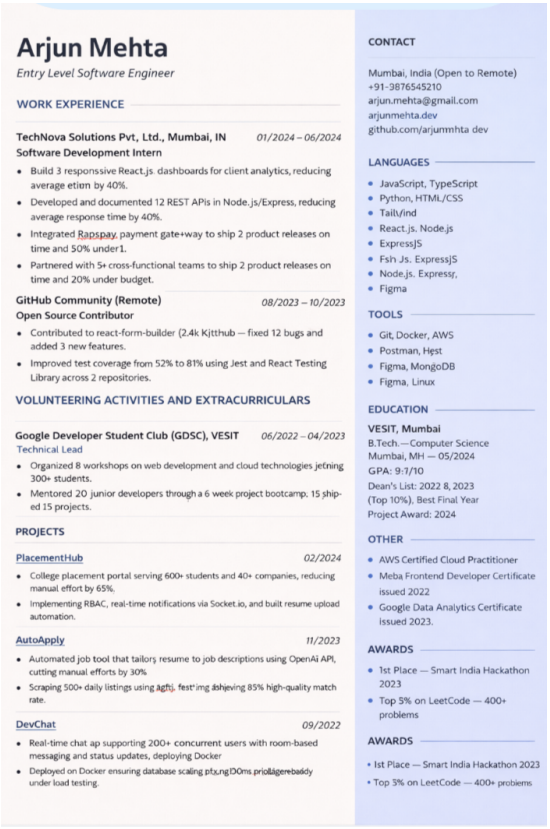


(c) Data Processing Stage

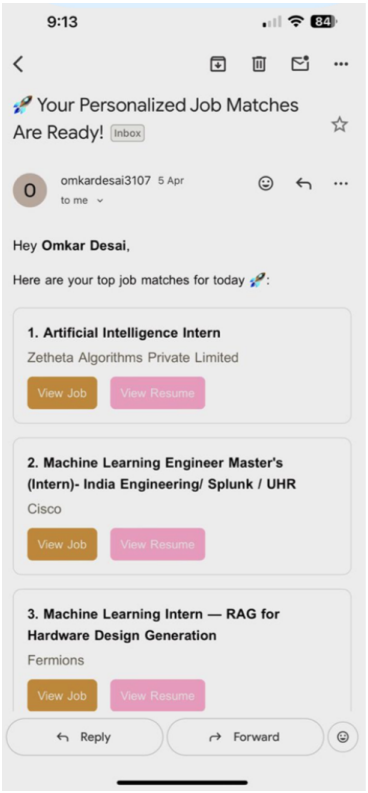


(d) User Portfolio View

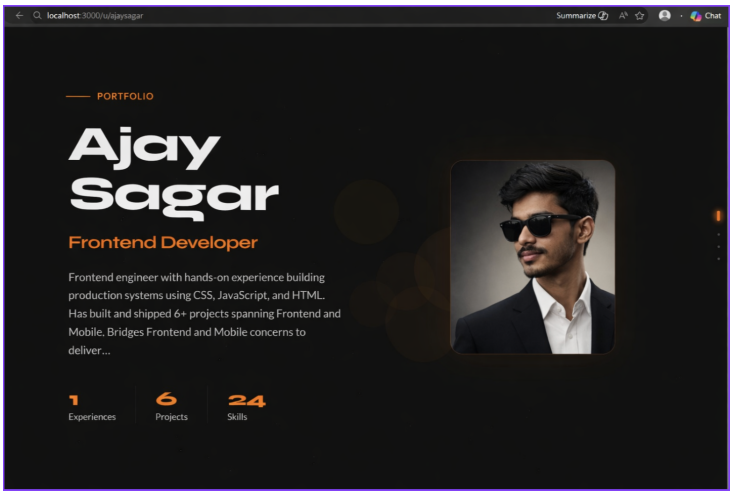
Figure 5.1: System Interface Screenshots (Part 1)



(a) Job Preferences Dashboard



(b) Generated Resume Output



(c) Job Matches Notification Interface

Figure 5.2: System Interface Screenshots (Part 2)

5.2 Sample Code (Implementation)

This section presents key implementation snippets from the proposed system, focusing on the most critical and technically significant components. These include the multi-agent resume generation pipeline, semantic job matching using vector similarity, and the evaluation mechanism used for performance analysis.

5.2.1 Multi-Agent RAG Orchestrator

The core of the system is a multi-agent pipeline based on a *Retriever–Drafter–Critic* architecture. The Drafter agent generates a resume, while the Critic agent evaluates it iteratively until it satisfies a predefined quality threshold.

Listing 5.1: Multi-Agent Resume Refinement Loop

```
let bestResume = null;
let bestAtsScore = 0;
let criticFeedback;

for (let iteration = 1; iteration <= maxIterations; iteration++) {
  const drafterResult = await draft({
    userProfile,
    jobDescription,
    examples,
    criticFeedback
  });

  if (!drafterResult.success) break;

  const criticResult = await critique({
    resume: drafterResult.resume,
    jobDescription,
    atsThreshold: 75
  });

  iterationScores.push(criticResult.atsScore);

  if (criticResult.approved) {
    bestResume = drafterResult.resume;
    bestAtsScore = criticResult.atsScore;
    break;
  }

  criticFeedback = criticResult.feedback;
}
```

5.2.2 Semantic Job Matching using FAISS

Semantic job matching is implemented using vector embeddings and FAISS, enabling similarity-based retrieval between candidate profiles and job descriptions.

Listing 5.2: FAISS-based Semantic Search

```python
import faiss
import numpy as np

class FAISSVectorStore:
    def __init__(self):
        self._dim = 384
        self._index = faiss.IndexFlatIP(self._dim)

    def search(self, query_vector, top_k=10):
        q = np.array([query_vector], dtype=np.float32)
        faiss.normalize_L2(q)
        scores, indices = self._index.search(q, top_k)
        return list(zip(indices[0], scores[0]))
```

5.2.3 Evaluation Module

An evaluation pipeline is implemented to measure system performance. It computes metrics such as Precision@K for job matching and tracks improvements in resume quality across iterations.

Listing 5.3: Evaluation Script

```javascript
const report = await generateEvaluationReport();

console.log(`
Precision: ${report.matching.precision}
Average ATS Score: ${report.resume.avg_score}
Improvement: ${report.resume.improvement}
`);
```

5.3 Evaluation and Discussion

5.3.1 Experimental Setup

The system was evaluated on **25 diverse user profiles** across domains such as web development, data science, and software engineering. A total of **130 end-to-end executions** were performed, including job matching, resume generation, and iterative refinement.

5.3.2 Job Matching Performance

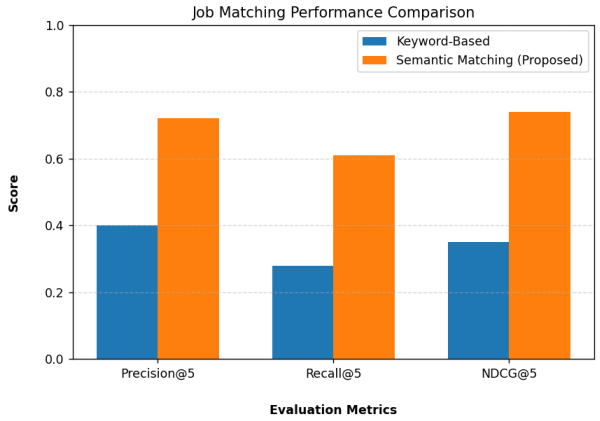


Figure 5.3: Job Matching Performance Comparison

Semantic matching outperforms keyword-based methods across **Precision@5, Recall@5, and NDCG@5**, showing better contextual understanding and ranking of relevant jobs.

5.3.3 Resume Quality

Table 5.1: Resume Quality Comparison

Method	**ATS Score**
Base Resume	62
Single-Pass	74
Proposed System	88

The proposed system significantly improves resume quality, mainly due to retrieval-based context and iterative refinement.

5.3.4 Iteration Analysis

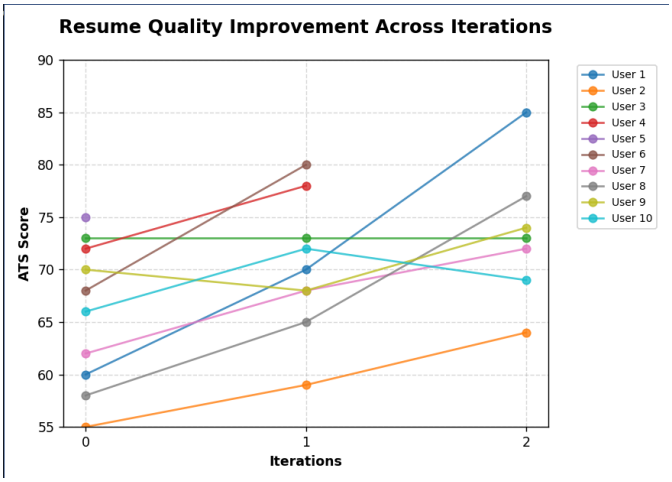


Figure 5.4: Resume Quality Across Iterations

Most users show improvement across iterations, while some stabilize early or show minor fluctuations. The system retains the **best-performing resume** to ensure reliability.

5.3.5 Ablation Study

Table 5.2: Ablation Results

Configuration	ATS Score
Without RAG	70
Without Refinement	65
Full System	88

Both retrieval and refinement are critical; removing them reduces performance significantly.

5.3.6 Discussion

The results demonstrate that combining semantic job matching, retrieval-augmented generation (RAG), and iterative refinement significantly improves both job relevance and resume quality. Semantic matching enables better identification of suitable job opportunities by capturing contextual relationships beyond simple keyword overlap, leading to more meaningful recommendations.

The integration of RAG allows the system to generate resumes that are tailored to specific job requirements by incorporating relevant contextual examples. Additionally, the iterative refinement process further enhances

resume quality through feedback-driven improvements, resulting in consistent performance gains across most user profiles. While the majority of profiles benefit from multiple iterations, some reach optimal quality early, indicating that refinement is adaptive rather than uniformly required.

The system also demonstrates stable behavior under different conditions, maintaining reliable output even when variations occur in input data. However, the overall performance is influenced by the quality and completeness of input profiles and retrieved contextual data. In summary, the proposed approach provides an effective and practical solution for automated, job-specific resume optimization, combining accuracy, adaptability, and scalability.

Chapter 6

Conclusion & Future Scope

6.1 Conclusion

This work presented **PlaceMates**, a unified system that integrates semantic job matching with retrieval-augmented resume generation and iterative refinement within a single pipeline. By representing candidate profiles and job descriptions in a shared semantic space, the system improves job recommendations beyond traditional keyword-based approaches and enables more accurate alignment between user capabilities and job requirements. Integration of retrieval-augmented generation allows the system to produce context-sensitive, job-specific resumes, while the iterative refinement mechanism enhances quality through structured, feedback-driven improvements.

Experimental results demonstrate improved job matching relevance and resume quality across diverse user profiles. The system adapts to different scenarios, maintains processing stability, and generates meaningful outputs under varying conditions. Overall, the proposed approach effectively combines semantic retrieval with iterative feedback-driven generation, offering a practical and scalable solution for personalized resume optimization.

6.2 Future Scope

While the results are promising, the current evaluation is based on a controlled dataset and may vary when deployed in real-world environments with dynamic and diverse user inputs. Future work will focus on extending the system to larger datasets, incorporating real user feedback, and improving validation mechanisms to better reflect real-world usage scenarios. Enhancing the robustness of the system under varying data quality and scaling conditions will also be an important direction.

Further improvements can include advanced retrieval strategies for better contextual grounding, along with personalization techniques that adapt resume generation based on individual user preferences and career goals. The system can also be extended to support real-time job data integration, continuous learning pipelines, and adaptive refinement strategies that evolve with changing job market trends. In addition, incorporating domain-specific customization and improved evaluation metrics can further enhance the applicability and performance of the system in different industries.

References

[1] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *EMNLP*, 2019.

[2] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *NeurIPS*, 2020.

[3] J. Malinowski *et al.*, "Matching people and jobs: A bilateral recommendation approach," in *EC-Web*, 2006.

[4] T. Gao *et al.*, "Simcse: Simple contrastive learning of sentence embeddings," *arXiv*, 2021.

[5] J. Johnson *et al.*, "Faiss: A library for efficient similarity search," *Facebook AI*, 2017.

[6] N. Thakur *et al.*, "Beir: A heterogeneous benchmark for information retrieval," in *NeurIPS*, 2021.

Acknowledgement

Success of a project like this involving high technical expertise, patience and massive support of guides, is possible when team members work together. We take this opportunity to express our gratitude to those who have been instrumental in the successful completion of this project. We would like to show our appreciation to **Mrs.Priyamvada S** for their tremendous support and help, without them this project would have reached nowhere. We would also like to thank our project coordinator **Dr. Chhaya Pawar** for providing us with regular inputs about documentation and project timeline. A big thanks to our HOD **Dr. Kiruthika M** for all the encouragement given to our team. We would also like to thank our principal, **Dr. S. M. Khot**, and our college, **Fr. C. Rodrigues Institute of Technology, Vashi**, for giving us the opportunity and the environment to learn and grow.

Project Group Members:

1. Omkar Desai, 1023125

_____

2. Kalpesh Eragi, 1023127

_____

3. Rohan Karkada, 1023148

_____

4. Shivangi Khandelwal, 1023152

_____

Appendix A : Timeline Chart

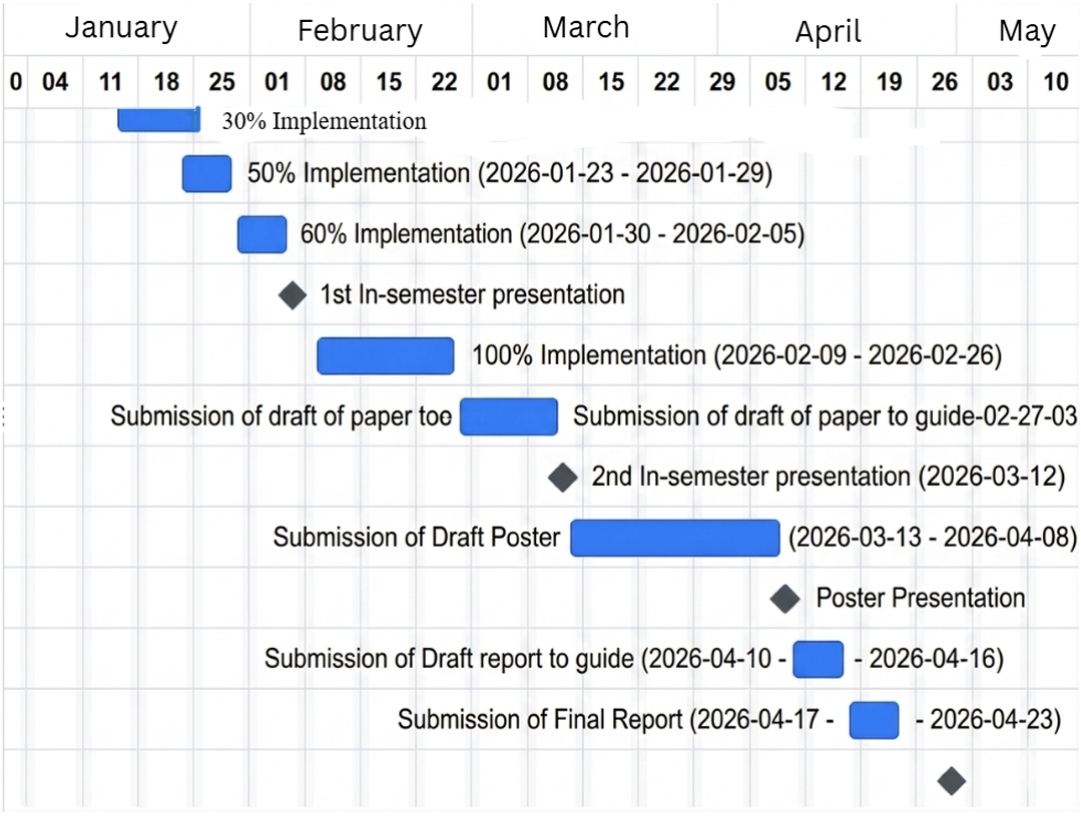


Figure 6.1: Project Timeline Chart

Appendix B: Publication Details

Paper Details

Title:
A Multi-Agent Retrieval-Augmented Framework for Semantic Job Matching and Iterative Resume Optimization

Authors:
Kalpesh Eragi , Omkar Desai , Rohan Karkada , Shivangi Khandelwal

Status:
Under Preparation

Target Venue: